SIMATS ENGINEERING

Department of Information Technology

CSA37 – Software Testing

ASSIGNMENT 1 – CO1

Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application

CO1: Testing viewed as an engineering activity, the role of process in software quality, testing as a process, basic definitions, principles of software testing, the role of the tester, origins of defects, defect classes, defect repository and defect examples.

Student Name	S Deepak	**Register Number**	1921260077
Department / Section	B.Tech IT	**Academic Year / Semester**	2026-2030
Faculty Name	R Kesavan	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	02-11-2026	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks
1	Evaluate Testing as an Engineering Activity	5		
2	Evaluate the Role of Process in Software Quality	2		
3	Evaluate Testing as a Process	6		
4	Apply and Critique Basic Definitions	4		
5	Evaluate the Application of Testing Principles	3		
6	Evaluate the Tester’s Role	3		
7	Trace the Origins of Defects	4		
8	Evaluate Defect Classes and Severity	4		
9	Design and Appraise a Defect Repository	3		
10	Evaluate Defect Examples and Prevention Strategies	4		
11	Recommend and Defend a Quality Improvement Plan	4		

Task	Task Title	Max Marks	Marks Awarded	Remarks
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of the Original Plan	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	**50**		**Percentage:**

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A reasoned evaluation of the MediQueue testing practice against the characteristics of an engineering activity, supported by evidence from the case.		
2	A justified position on the role of process in quality, supported by the case data.		
3	A process-based evaluation of testing in MediQueue, supported by cost calculations from the defect data and a reasoned estimate of savings.		
4	Correct application of basic testing definitions to the MediQueue incidents and a justified judgement on the effect of terminology on quality.		
5	A justified evaluation of the developers' claim using testing principles linked to evidence from the case.		
6	A defended judgement on the value of an independent tester for MediQueue, linked to specific incidents.		
7	A justified origin for each incident and an interpretation of the overall pattern of defect origins.		
8	A justified classification of all ten incidents by class, severity and priority, with a recommendation on which class to target first.		
9	A defect repository design for MediQueue with a justification for each field and an evaluation of its benefits.		
10	A root-cause analysis of three defects with justified, cost-aware prevention strategies.		
11	A technically justified quality improvement plan supported by the evidence developed in earlier tasks.		
12A–D	Impact analysis, evaluation of the original solution, adaptation or redesign, and evidence-based defence		

DEFECT ANALYSIS EVIDENCE CHAIN (Tasks 3, 7–10)

Evidence item	Status	Observation
Defect data calculations (cost per phase, totals)		
Phase-containment / detection figures		
Origin assigned to every incident (I1–I10)		
Class, severity and priority justified		
Defect repository fields with reasons		
Root cause → prevention strategy link		
Cost–benefit of prevention strategies		

Evidence item	Status	Observation
Interpretation of the defect pattern		
Figures and conclusions consistent		

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 12 tasks (including 12A–12D) answered	☐ Yes ☐ Partly ☐ No
Defect cost calculations shown step by step	☐ Yes ☐ Partly ☐ No
All ten incidents classified (origin, class, severity, priority)	☐ Yes ☐ Partly ☐ No
Error, fault and failure separated for I3, I5 and I6	☐ Yes ☐ Partly ☐ No
Testing principles linked to evidence from the case	☐ Yes ☐ Partly ☐ No
Repository design justified field by field	☐ Yes ☐ Partly ☐ No
Tables, figures and diagrams included and readable	☐ Yes ☐ Partly ☐ No
Cross-references and table/figure numbering correct	☐ Yes ☐ Partly ☐ No
Work consistent throughout	☐ Yes ☐ Partly ☐ No
No unfilled template placeholders	☐ Yes ☐ Partly ☐ No

Software Testing (CSA37)
ASSIGNMENT 1

CO1 Coverage: Testing viewed as an engineering activity, the role of process in software quality, testing as a process, basic definitions, principles of software testing, the role of the tester, origins of defects, defect classes, defect repository and defect examples.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application (Total Marks - 50)

Duration:1week

Scenario — Hospital Outpatient Appointment Application (MediQueue)

A multi-speciality hospital launched MediQueue, a mobile and web application that allows patients to book outpatient appointments, pay consultation fees, receive token numbers and view prescriptions. About 6,000 patients use the application every day.

The application was built in eight months by a team of five developers. There was no separate tester. Developers "tested" their own code just before release, and the release was approved when the demo to the hospital management showed no errors.

Within three months of release, the hospital received many complaints. The management has asked you, as a newly appointed Quality Engineer, to investigate why so many defects reached patients and to recommend how quality should be built into the process.

Incidents reported after release:

ID	Incident
I1	Two patients receive the same token number for the same doctor when they book at the same second.
I2	Appointments for Sunday are accepted although the outpatient department is closed on Sundays; this rule was never written in the requirements.
I3	Consultation fee shown as ₹500 on the booking page but ₹550 is charged; the GST rule was misunderstood.
I4	Application crashes when a patient name has more than 40 characters.
I5	Senior-citizen priority is not applied for patients who turn 60 on the day of the appointment.
I6	Prescription PDF shows the previous patient's details when two doctors save at the same time.
I7	The user manual tells patients to cancel through "My Visits", but this menu does not exist.
I8	A test case for refund was marked "Pass" because the tester compared the output with a wrong expected value.
I9	SMS reminders are not sent when the hospital's SMS provider changes its API key.
I10	The doctor's schedule screen takes 25 seconds to load on the hospital's Wi-Fi network.

Defect data for the last release (100 defects):

Phase	Defects introduced	Defects detected	Cost to fix per defect (₹)	Remarks
Requirements	30	2	500	No formal review
Design	22	3	1,500	No formal review
Coding	38	15	3,000	Developer self-check only
Testing	5	20	8,000	Demo-based testing
Documentation	5	0	–	–
Production (after release)	–	60	40,000	Reported by patients

Design Challenge:

The hospital now plans to release a new version every two weeks and to connect MediQueue to the national digital health ID system.

Re-evaluate your recommended quality process and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate Testing as an Engineering Activity 5 Marks
CO1 | Bloom's Level: K5 – Evaluate

Investigate whether the way MediQueue was tested can be called an engineering activity.

Did the MediQueue team engineer quality into the product, or did they only hope for it?

Discuss:

- **The characteristics of testing as an engineering activity (planning, measurement, standards, repeatability)**
- **Which of these characteristics were missing in the MediQueue project, with evidence from the case**
- **How the absence of each characteristic contributed to the incidents reported**
- **What an engineered testing approach would have looked like for this project**

You are expected to judge the project against engineering practice, not merely define the term.

Evidence Expected
A reasoned evaluation of the MediQueue testing practice against the characteristics of an engineering activity, supported by evidence from the case.

TASK 2 – Evaluate the Role of Process in Software Quality 2 Marks
CO1 | Bloom's Level: K5 – Evaluate

The hospital management believes that hiring "better developers" will solve the quality problem.

Is quality mainly a result of good people, or of a good process?

Discuss:

- **The role of a defined process in achieving software quality**
- **Whether better developers alone would have prevented the incidents**
- **How a process maturity view (for example, TMM levels) would place the current team**

Your position must be defended with evidence from the case.

Evidence Expected
A justified position on the role of process in quality, supported by the case data.

TASK 3 – Evaluate Testing as a Process 6 Marks
CO1 | Bloom's Level: K5 – Evaluate

In MediQueue, testing was a single activity done just before release.

What would change in the defect data if testing had been treated as a process running through the whole life cycle?

Discuss:

- **Testing as a process with planning, design, execution, evaluation and reporting activities**
- **The relationship between verification and validation activities and each development phase**
- **The phase in which each type of MediQueue defect should have been detected**
- **Using the defect data table, the cost the hospital paid because defects were found late**
- **The likely saving if requirements and design reviews had been conducted**
- **The limits of your estimate and the assumptions it depends on**

You must show your calculations and interpret them. Stating that "early testing is cheaper" will not receive full credit.

Evidence Expected
A process-based evaluation of testing in MediQueue, supported by cost calculations from the defect data and a reasoned estimate of savings.

TASK 4 – Apply and Critique Basic Definitions 4 Marks
CO1 | Bloom's Level: K5 – Evaluate

The team uses the words "bug", "error" and "failure" interchangeably in its reports.

Does careless terminology actually harm quality, or is it only a matter of words?

Discuss:

- **The meaning of error, fault (defect), failure, test case, test oracle and test bed in the MediQueue context**
- **For incidents I3, I5 and I6, the error, the fault and the failure**
- **Why the confusion in terminology made incident I8 possible**
- **Whether the team needs a common glossary, and what it must contain**

Definitions copied from a textbook will not receive full credit. Each must be applied to MediQueue.

Evidence Expected
Correct application of basic testing definitions to the MediQueue incidents and a justified judgement on the effect of terminology on quality.

TASK 5 – Evaluate the Application of Testing Principles 3 Marks
CO1 | Bloom's Level: K5 – Evaluate

The developers claim that "we tested everything and found no errors, so the software was correct".

Which testing principles prove this claim wrong?

Discuss:

- **At least four software testing principles that apply to the MediQueue situation**
- **For each principle, the incident or decision in the case that violates or confirms it**
- **Which principle, if followed, would have had the largest effect on quality, and why**

Do not simply list principles. You are expected to relate each principle to evidence.

Evidence Expected
A justified evaluation of the developers' claim using testing principles linked to evidence from the case.

TASK 6 – Evaluate the Tester's Role 3 Marks
CO1 | Bloom's Level: K5 – Evaluate

MediQueue had no independent tester.

Would an independent tester have changed the outcome, or would the same defects still have reached patients?

Discuss:

- **The role and responsibilities of a tester across the life cycle**

- **The advantages and limitations of developers testing their own code**
- **Which incidents a tester would most likely have caught, and at which stage**
- **The qualities a tester for a healthcare application must have**

Your answer must take a position and defend it.

Evidence Expected

A defended judgement on the value of an independent tester for MediQueue, linked to specific incidents.

TASK 7 – Trace the Origins of Defects 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Using the incidents reported after release, investigate where each defect originated.

Where in the life cycle are MediQueue's defects really coming from?

Discuss:

- **The origin of each incident: requirements, design, code, testing, documentation or environment**
- **The evidence in the case that supports each origin you assign**
- **Incidents that may have more than one origin, and how you decided**
- **What the overall pattern of origins tells the hospital about where to invest effort**

Where an incident could belong to more than one origin, argue for your choice.

Evidence Expected

A justified origin for each incident and an interpretation of the overall pattern of defect origins.

TASK 8 – Evaluate Defect Classes and Severity 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

A defect classification scheme helps the team understand the nature of its defects.

Which classes of defects are most harmful to patients, and do the team's priorities reflect that?

Discuss:

- **A suitable defect classification (requirements/specification, design, coding, testing defects and their sub-classes)**
- **The class, severity and priority of each incident, with reasons**
- **Why severity and priority may differ for the same defect (for example, I7 and I1)**
- **Which defect class the team should target first, and why**

Simply placing defects in a table is not sufficient. You are expected to justify each classification.

Evidence Expected

A justified classification of all ten incidents by class, severity and priority, with a recommendation on which class to target first.

TASK 9 – Design and Appraise a Defect Repository 3 Marks
CO1 | Bloom's Level: K5 – Evaluate

The hospital currently records complaints in e-mails and a spreadsheet.

What information must a defect repository hold so that the team can learn from its defects and stop repeating them?

Discuss:

- **The fields a defect repository for MediQueue must contain, and why each is needed**
- **How the repository supports defect analysis, process improvement and prevention**
- **The weaknesses of the current e-mail and spreadsheet approach**

You are expected to justify your repository design, not just list fields.

Evidence Expected

A defect repository design for MediQueue with a justification for each field and an evaluation of its benefits.

TASK 10 – Evaluate Defect Examples and Prevention Strategies 4 Marks
CO1 | Bloom's Level: K5 – Evaluate

Select three incidents that represent different defect classes.

For each selected defect, what single change to the process would have prevented it?

Discuss:

- **The root cause of each selected defect**
- **A prevention strategy for each (for example, requirement reviews, design inspection, coding standards, test oracles)**
- **The cost and effort of each strategy compared with its benefit**
- **Which strategy gives the best value for the hospital**

Your recommendation must be supported by evidence from the case and the defect data.

Evidence Expected

A root-cause analysis of three defects with justified, cost-aware prevention strategies.

TASK 11 – Recommend and Defend a Quality Improvement Plan 4 Marks
CO1 | Bloom's Level: K5 – Evaluate

Based on your analysis in Tasks 1–10, recommend a quality improvement plan for MediQueue.

Discuss:

- **The process changes you recommend and their order of priority**
- **The role of testers, reviews and the defect repository in your plan**
- **The expected effect on the number and cost of defects**
- **Trade-offs, risks and limitations of your plan**
- **Whether the plan remains practical for a small team of five developers**

Your recommendation must follow from your own analysis. A generic list of best practices is not sufficient.

Evidence Expected

A technically justified quality improvement plan supported by the evidence developed in earlier tasks.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Quality Process 8 Marks
CO1 | Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital now wants to release a new version of MediQueue every two weeks instead of every six months.

At the same time, MediQueue must connect to the national digital health ID system, which is maintained by an external agency.

Patient health records will now be shared between hospitals.

Re-evaluate your original solution in response to these changed conditions.

You may: RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis – 2 Marks
CO1 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed from the original situation**
- **Which assumptions in your quality improvement plan are affected**
- **Why frequent releases change the role of testing and the tester**
- **Why sharing health records raises the severity of some defect classes**

Evidence Expected

A clear analysis of how the changed conditions affect the original quality improvement plan.

B. Evaluation of the Original Plan – 2 Marks
CO1 | Bloom's Level: K5 – Evaluate

Evaluate whether your recommended plan remains appropriate.

Discuss:

- **Whether the planned reviews and test activities fit into a two-week cycle**
- **Which new defect origins are introduced by the external health ID system**
- **Whether your defect repository is still sufficient**

Evidence Expected

A technically supported evaluation of the suitability of the original plan under the new conditions.

C. Adaptation or Redesign – 2 Marks
CO1 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the quality process.

If you modify or redesign it:

- **Describe the revised process and the tester's revised role**
- **Explain which testing principles now become more important**
- **Explain how defects from the external system will be classified and tracked**

If you retain the original plan, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate response to the changed requirement with clear quality-engineering reasoning.

D. Evidence-Based Defence – 2 Marks
CO1 | Bloom's Level: K5 – Evaluate

Defend your final decision using evidence.

Use:

- **The defect data and cost figures**
- **Defect origins and classes**
- **Release frequency**
- **Patient-safety and data-privacy risk**
- **Team size and capacity**

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final decision integrating defect evidence, process reasoning and the changed constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Evaluate Testing as an Engineering Activity	5
Task 2 – Evaluate the Role of Process in Software Quality	2
Task 3 – Evaluate Testing as a Process	6
Task 4 – Apply and Critique Basic Definitions	4
Task 5 – Evaluate the Application of Testing Principles	3
Task 6 – Evaluate the Tester's Role	3
Task 7 – Trace the Origins of Defects	4
Task 8 – Evaluate Defect Classes and Severity	4
Task 9 – Design and Appraise a Defect Repository	3
Task 10 – Evaluate Defect Examples and Prevention Strategies	4
Task 11 – Recommend and Defend a Quality Improvement Plan	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	50

Software Testing (CSA37)
ASSIGNMENT 1

CO1 Coverage: Testing viewed as an engineering activity, the role of process in software quality, testing as a process, basic definitions, principles of software testing, the role of the tester, origins of defects, defect classes, defect repository and defect examples.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application (Total Marks - 50)

Duration:1week

Scenario — Hospital Outpatient Appointment Application (MediQueue)

A multi-speciality hospital launched MediQueue, a mobile and web application that allows patients to book outpatient appointments, pay consultation fees, receive token numbers and view prescriptions. About 6,000 patients use the application every day.

The application was built in eight months by a team of five developers. There was no separate tester. Developers "tested" their own code just before release, and the release was approved when the demo to the hospital management showed no errors.

Within three months of release, the hospital received many complaints. The management has asked you, as a newly appointed Quality Engineer, to investigate why so many defects reached patients and to recommend how quality should be built into the process.

Incidents reported after release:

ID	Incident
I1	Two patients receive the same token number for the same doctor when they book at the same second.
I2	Appointments for Sunday are accepted although the outpatient department is closed on Sundays; this rule was never written in the requirements.
I3	Consultation fee shown as ₹500 on the booking page but ₹550 is charged; the GST rule was misunderstood.
I4	Application crashes when a patient name has more than 40 characters.
I5	Senior-citizen priority is not applied for patients who turn 60 on the day of the appointment.
I6	Prescription PDF shows the previous patient's details when two doctors save at the same time.
I7	The user manual tells patients to cancel through "My Visits", but this menu does not exist.
I8	A test case for refund was marked "Pass" because the tester compared the output with a wrong expected value.
I9	SMS reminders are not sent when the hospital's SMS provider changes its API key.
I10	The doctor's schedule screen takes 25 seconds to load on the hospital's Wi-Fi network.

Defect data for the last release (100 defects):

Phase	Defects introduced	Defects detected	Cost to fix per defect (₹)	Remarks
Requirements	30	2	500	No formal review
Design	22	3	1,500	No formal review
Coding	38	15	3,000	Developer self-check only
Testing	5	20	8,000	Demo-based testing
Documentation	5	0	–	–
Production (after release)	–	60	40,000	Reported by patients

Design Challenge:

The hospital now plans to release a new version every two weeks and to connect MediQueue to the national digital health ID system.

Re-evaluate your recommended quality process and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate Testing as an Engineering Activity 5 Marks
CO1 | Bloom's Level: K5 – Evaluate

Investigate whether the way MediQueue was tested can be called an engineering activity.

Did the MediQueue team engineer quality into the product, or did they only hope for it?

Discuss:

- **The characteristics of testing as an engineering activity (planning, measurement, standards, repeatability)**
- **Which of these characteristics were missing in the MediQueue project, with evidence from the case**
- **How the absence of each characteristic contributed to the incidents reported**
- **What an engineered testing approach would have looked like for this project**

You are expected to judge the project against engineering practice, not merely define the term.

Evidence Expected
A reasoned evaluation of the MediQueue testing practice against the characteristics of an engineering activity, supported by evidence from the case.

Ans:

Was the MediQueue Testing an Engineering Activity?

My position is that it was not. The MediQueue team hoped for quality; they did not engineer it. Engineering means that the work is planned, measured, done to agreed standards and repeatable, so that a result can be explained and obtained again. The only evidence of quality the team had was one demonstration in which nobody saw an error, and the 100-defect record shows how little that was worth: 40 defects were found before release and 60 were found by patients.

1. The four characteristics and the MediQueue practice

Table 1: Characteristics of an engineering activity compared with MediQueue practice

Characteristic	What engineering practice requires	What MediQueue did (evidence from the case)	Incidents it contributed to
Planning	Test goals, scope, risks, schedule and exit criteria are written down before testing starts	No plan and no tester. Developers "tested" their own code shortly before release	I1, I6 (concurrency was never named as a risk); I10 (no performance target)
Measurement	Defects, coverage and effort are counted per phase and compared with a target	Release was approved because the demo "showed no errors". No defect counts existed before release; the figures were only assembled after 60 defects had reached patients	All ten; I8 (a pass verdict that nobody measured against the right value)
Standards	Agreed techniques, review checklists, coding standards, entry and exit criteria	The remarks column of the defect data reads "No formal review", "Developer self-check only" and "Demo-based testing"	I2, I3 (no requirement review); I4, I5 (no boundary-value standard)
Repeatability	Tests are written down, kept and can be rerun by someone else with the same result	Only a demonstration; no stored test cases, no test environment. An SMS key change (I9) or a slow network (I10) could not be checked again because nothing existed to repeat	I8, I9, I10

2. How each missing characteristic contributed to the incidents

- **No planning** meant nobody asked "what could go wrong with 6,000 patients a day?". Simultaneous actions (I1, I6) and speed on a hospital network (I10) are exactly the risks a test plan lists; a demo with one user at a time cannot reveal them.
- **No measurement** meant the team could not see that its testing was weak. In the data, the testing phase received 75 defects and found 20 of them (26.7%), the coding phase found 15 of 85 (17.6%), and requirements and design found only 5 of 52 (9.6%). Nobody saw these figures until patients complained.
- **No standards** meant that rules existed only in people's heads. The Sunday rule (I2) was never written, and the GST rule (I3) was read differently by different people. A review checklist item such as "every business rule has a written example" would have exposed both.
- **No repeatability** meant that every fix was untested for side effects and the refund test (I8) was passed on a wrong expected value that nobody could compare with a source.

3. What an engineered testing approach would have looked like

Table 2: An engineered test approach for MediQueue

Activity	What would have been done	Output	Incidents it would have addressed
Test planning	Identify risks (concurrency, money, privacy, performance, external services); set levels, schedule and exit criteria (for example, no open critical defect)	Test plan	I1, I6, I10
Review of requirements	Checklist review with hospital staff: working days, fee and GST rule with worked examples, field limits, response times	Reviewed requirements with examples	I2, I3, I4, I10
Test design	Boundary tests (name 40/41/42 characters; age 59/60/61), two bookings in the same second, fee page compared with the payment	Test cases with expected results from a named source	I1, I3, I4, I5, I6
Test environment	Test doctors and patients, an SMS sandbox, a realistic Wi-Fi network	Test bed	I9, I10
Execution and measurement	Run at unit, integration and system level; record pass or fail; compare with exit criteria	Test log and defect reports	I7, I8
Regression	Rerun the stored tests after every fix	Stored, repeatable test suite	All

Conclusion: the project had one of the four characteristics in name only (a demo is an event, not a repeatable test), and none of them in substance. The data in Table 1 supports the judgement that the team hoped for quality and was lucky for the 40 defects it caught before release.

TASK 2 – Evaluate the Role of Process in Software Quality 2 Marks
CO1 | Bloom's Level: K5 – Evaluate

The hospital management believes that hiring "better developers" will solve the quality problem.

Is quality mainly a result of good people, or of a good process?

Discuss:

- **The role of a defined process in achieving software quality**
- **Whether better developers alone would have prevented the incidents**
- **How a process maturity view (for example, TMM levels) would place the current team**

Your position must be defended with evidence from the case.

Evidence Expected
A justified position on the role of process in quality, supported by the case data.

Ans:

Is Quality the Result of People or of Process?

My position is that quality is mainly the result of a good process. Good people are necessary, but the incident list and the defect data show that most of the failures were outside the reach of any individual developer, however skilled.

1. The role of a defined process

A defined process turns good intentions into repeated, checkable steps: someone reviews what the analyst wrote, someone other than the author checks the code, tests are planned, and results are recorded. It catches mistakes that even excellent people make (a skilled developer can still misread the GST rule) and it works at every phase, not only in code.

2. Would better developers alone have prevented the incidents?

Table 3: Could a better developer alone have prevented each incident?

Incident	Prevented by a better developer alone?	Reason
I4, I5	Yes	Input-length checks and boundary comparisons are coding skills
I1, I6, I10	Partly	Experienced developers think about concurrency and speed, but without a design review and a test on a real network such defects still slip through
I2, I3	No	The Sunday rule was never given to the developer and the GST rule was misread; coding skill cannot supply a rule that was missing or wrong
I7	No	A documentation defect; it needs a check of the manual against the build
I8	No	A defect in testing; it needs a review of test cases and an independent tester
I9	No	The SMS provider changed its key; this needs interface monitoring and configuration management

Skill alone could have prevented at most 2 incidents with certainty (I4, I5) and might have helped with 3 more (I1, I6, I10). The defect data agrees: only 38 of the 100 defects were introduced during coding; 52 came from requirements and design. Even if better developers halved the coding defects, 19 fewer defects would be introduced and 81 would remain; if they removed coding defects altogether, 62 would remain. “Hiring better developers” therefore addresses less than two-fifths of the problem at best.

3. Where the team stands on a process maturity scale

Using the Testing Maturity Model (TMM), I place the team at Level 1 (Initial). At this level testing is not separated from debugging, the aim of testing is only to show that the software works (the demo), and there is neither a test plan nor a trained tester. The team should aim for Level 2 (Phase Definition), whose maturity goals match exactly what is missing: develop testing and debugging goals, start a test planning process and institutionalise basic testing techniques and methods. Level 3 (Integration) comes after that: a test group, testing integrated into the life cycle, and training.

TASK 3 – Evaluate Testing as a Process 6 Marks
CO1 | Bloom's Level: K5 – Evaluate

In MediQueue, testing was a single activity done just before release.

What would change in the defect data if testing had been treated as a process running through the whole life cycle?

Discuss:

- **Testing as a process with planning, design, execution, evaluation and reporting activities**
- **The relationship between verification and validation activities and each development phase**
- **The phase in which each type of MediQueue defect should have been detected**
- **Using the defect data table, the cost the hospital paid because defects were found late**
- **The likely saving if requirements and design reviews had been conducted**
- **The limits of your estimate and the assumptions it depends on**

You must show your calculations and interpret them. Stating that "early testing is cheaper" will not receive full credit.

Evidence Expected

A process-based evaluation of testing in MediQueue, supported by cost calculations from the defect data and a reasoned estimate of savings.

Ans:

Testing as a Process Across the Life Cycle

In MediQueue, testing happened as one event before release. I first outline what testing as a process would have involved, then identify the phase in which each incident should have been caught, and finally use the defect data to calculate what the late discovery cost and what review activities would have saved.

1. Testing viewed as a process

Table 4: Test process activities as applied to MediQueue

Activity	What it would involve in MediQueue	Output
Planning	Define test goals, levels, risks (concurrency, money, privacy), schedule and exit criteria	Test plan
Design	Derive test cases from the requirements: age 59/60/61, name length 40/41/42, Sunday booking, fee shown = fee charged, two bookings in the same second	Test cases with expected results
Set-up	A test bed with test doctors, patients, an SMS sandbox and a representative Wi-Fi network	Test environment
Execution	Run the tests at unit, integration and system level; rerun them after every fix	Test log
Evaluation	Compare actual with expected results (the oracle); decide pass or fail; check the exit criteria	Verdicts and defect reports
Reporting and closure	Defect report, test summary and lessons learnt stored in the repository	Summary report and repository entries

2. Verification and validation for each phase

Table 5: Verification and validation activities for each development phase

Phase	Verification (are we building it right?)	Validation (are we building the right thing?)	Incidents targeted
Requirements	Requirement review with a checklist and a testability check	Acceptance tests planned with hospital staff	I2, I3, I10
Design	Design inspection (concurrency, data handling, interfaces)	System test cases designed	I1, I6, I9
Coding	Code review, coding standard and unit tests using boundary values	—	I4, I5
Testing	Review of test cases and expected results by a second person	System and acceptance tests on a realistic environment	I8, I10
Documentation	Review of the manual against the build	Walk-through of the manual by a new user	I7
Release and operation	Configuration checks of external interfaces	Monitoring in production	I9

3. The phase in which each incident should have been detected

Table 6: Phase in which each incident should have been detected

Incident	Origin	Should have been detected in	By which activity
I1	Design	Design	Design inspection; concurrency test at system level
I2	Requirements	Requirements	Requirement review (which days does the outpatient department work?)
I3	Requirements	Requirements	Review with the accounts staff; acceptance test of the fee
I4	Coding	Coding	Unit test with name lengths 40, 41 and 42
I5	Coding	Coding	Unit test with ages 59, 60 and 61
I6	Design	Design	Design inspection; concurrent save test
I7	Documentation	Testing	Document review; walk-through of the manual
I8	Testing	Testing	Review of test cases and their oracles
I9	Environment	Release / operation	Interface contract test; monitoring
I10	Requirements	Requirements, then system test	Response-time requirement; test on the hospital network

4. The cost the hospital paid because defects were found late

The cost of each phase is the number of defects detected there multiplied by the cost to fix one defect in that phase.

Table 7: Cost of defects by the phase in which they were detected

Phase of detection	Defects detected	Cost per defect (₹)	Calculation	Total cost (₹)	Share of total
Requirements	2	500	2 × 500	1,000	0.04 %
Design	3	1,500	3 × 1,500	4,500	0.17 %

Phase of detection	Defects detected	Cost per defect (₹)	Calculation	Total cost (₹)	Share of total
Coding	15	3,000	15 × 3,000	45,000	1.72 %
Testing	20	8,000	20 × 8,000	1,60,000	6.13 %
Production	60	40,000	60 × 40,000	24,00,000	91.94 %
Total	**100**			**26,10,500**	**100 %**

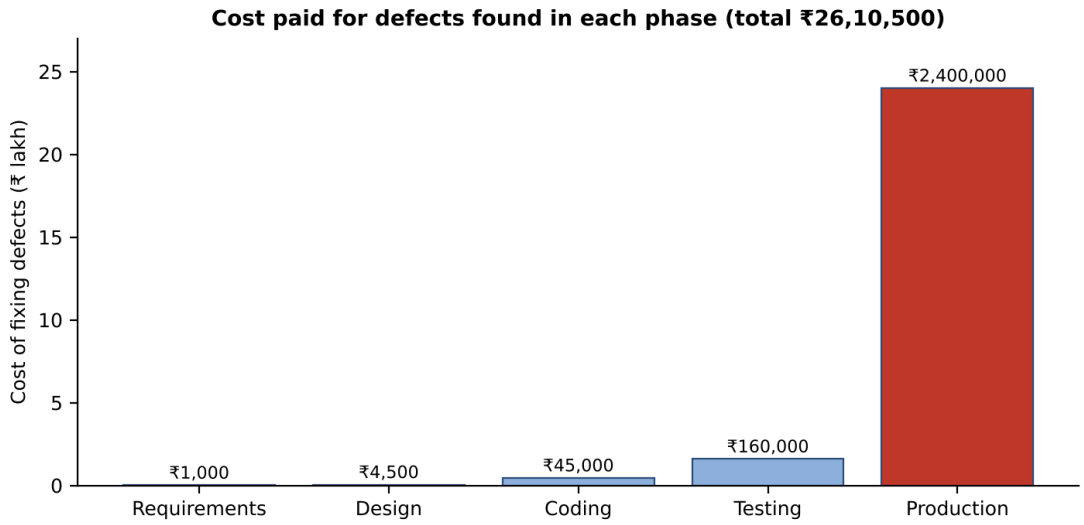


Figure 1: Cost paid for defects detected in each phase

- **Interpretation:** 60 of the 100 defects (60 %) were found by patients, and these 60 defects account for ₹24,00,000, which is 91.9 % of the total bill. The 40 defects found before release cost ₹2,10,500 together, an average of ₹5,263 each, whereas each production defect cost ₹40,000, which is 7.6 times that average.
- A defect fixed in production costs 80 times a requirement defect (40,000 ÷ 500), 26.7 times a design defect (40,000 ÷ 1,500), 13.3 times a coding defect and 5 times a testing defect.

Table 8: Defect containment by phase (defects present, found and escaping)

Phase	Present (carried + introduced)	Found in the phase	Escaping to the next phase	Share found
Requirements	0 + 30 = 30	2	28	6.7 %
Design	28 + 22 = 50	3	47	6.0 %
Coding	47 + 38 = 85	15	70	17.6 %
Testing	70 + 5 = 75	20	55	26.7 %
Documentation and release	55 + 5 = 60	0	60 reach patients	0 %

Of the 52 defects introduced in requirements and design, only 5 (9.6 %) were found before coding began. Overall, 40 % of the defects were caught before release and 60 % were not.

5. How much of the cost was avoidable?

If every defect had been found in the phase in which it was introduced, the cost would have been:

Table 9: Cost paid compared with the cost of finding each defect in its own phase

Phase of origin	Defects introduced	Cost per defect (₹)	Cost if found at origin (₹)
Requirements	30	500	15,000
Design	22	1,500	33,000
Coding	38	3,000	1,14,000
Testing	5	8,000	40,000
Documentation	5	500 (assumed; none given)	2,500
Total	**100**		**2,04,500**

Late detection cost ₹26,10,500 − ₹2,04,500 = **₹24,06,000**, which is 92.2 % of the bill. The ideal is not reachable, because no process finds every defect where it is created, but it shows the size of the opportunity.

6. The likely saving if requirements and design reviews had been held

I assume that a review finds 60 % of the defects present in the work product at the time (requirements, then design, which includes the requirement defects that escaped), and that the later phases detect the same share of what reaches them as before (17.6 % in coding, 26.7 % in testing). Under these assumptions:

Table 10: Defect flow and cost with requirement and design reviews

Phase	Present	Detected	Cost per defect (₹)	Cost (₹)	Escaping
Requirements (review 60 %)	30	18.0	500	9,000	12.0
Design (review 60 %)	12 + 22 = 34	20.4	1,500	30,600	13.6
Coding (17.6 %)	13.6 + 38 = 51.6	9.1	3,000	27,318	42.5
Testing (26.7 %)	42.5 + 5 = 47.5	12.7	8,000	1,01,321	34.8
Production	34.8 + 5 = 39.8	39.8	40,000	15,93,161	—
Total		**100**		**17,61,399**	

- **Saving:** ₹26,10,500 − ₹17,61,399 = **₹8,49,101 (32.5 %)**. The number of defects reaching patients falls from 60 to about 40.
- The reviews themselves have a cost. At an assumed 56 hours of effort (24 for requirements, 32 for design) and ₹500 per hour this is ₹28,000, so the net saving is about **₹8,21,000**.
- **Sensitivity:** if reviews found only 40 % of the defects the saving would be about ₹5,83,700; at 80 % it would be about ₹10,54,600.

7. Limits of the estimate and the assumptions it depends on

- The 60 % review efficiency is an assumption, not a measurement from this team; the sensitivity range above shows its effect.
- The later phases are assumed to detect the same share as before, which ignores the fact that they would receive fewer and easier defects.
- The cost per defect is taken as fixed for each phase, although a requirement defect found in production probably costs more than an average defect found there.
- The estimate covers one release and the money cost only: it does not include harm to patients, loss of trust or legal exposure, which are larger for incidents like I6.
- The cost of ₹500 per hour and the cost of fixing documentation defects (not given) are assumed.
- Fractions of defects (such as 20.4) are expected values, not counts that can occur.

TASK 4 – Apply and Critique Basic Definitions 4 Marks
CO1 | Bloom's Level: K5 – Evaluate

The team uses the words "bug", "error" and "failure" interchangeably in its reports.

Does careless terminology actually harm quality, or is it only a matter of words?

Discuss:

- **The meaning of error, fault (defect), failure, test case, test oracle and test bed in the MediQueue context**
- **For incidents I3, I5 and I6, the error, the fault and the failure**
- **Why the confusion in terminology made incident I8 possible**
- **Whether the team needs a common glossary, and what it must contain**

Definitions copied from a textbook will not receive full credit. Each must be applied to MediQueue.

Evidence Expected
Correct application of basic testing definitions to the MediQueue incidents and a justified judgement on the effect of terminology on quality.

Ans:

Do Careless Definitions Harm Quality?

My position is that careless terminology does harm quality. The three words "bug", "error" and "failure" name three different things (a mistake made by a person, the incorrect piece of the product that results, and the wrong behaviour that someone sees). If the team uses them interchangeably, reports cannot say where the problem lives, fixes are aimed at the symptom and the cause is left in place.

1. The basic definitions applied to MediQueue

Table 11: Basic definitions in the MediQueue context

Term	Meaning	MediQueue example
Error	A mistake made by a person	The developer misunderstood the GST rule; the analyst forgot to write that the outpatient department is closed on Sundays
Fault (defect)	The incorrect part of a work product that results from an error	The fee calculation that leaves out GST; the missing Sunday check in the booking code
Failure	Behaviour of the running system that departs from what is expected	The patient sees ₹500 and is charged ₹550; a Sunday appointment is accepted
Test case	Inputs, conditions and the expected result, written before running	Patient aged exactly 60 on the appointment date, booked a day earlier; expected: senior-citizen priority is applied
Test oracle	The source from which the expected result is obtained	The hospital's GST and refund rules signed off by the accounts department; the senior-citizen policy
Test bed	The environment in which the test is run	A test server with test doctors and patients, an SMS sandbox and a Wi-Fi link that behaves like the hospital's

2. Error, fault and failure for incidents I3, I5 and I6

Table 12: Error, fault and failure in incidents I3, I5 and I6

Incident	Error (human mistake)	Fault (in the product)	Failure (what happened)
I3	The analyst or developer misread how GST applies to the consultation fee	The booking page shows the fee without GST while the payment step adds it (two calculations of the same fee)	The patient sees ₹500 and is charged ₹550
I5	The age rule "senior citizen = 60 or more" was applied as "more than 60" (or the age was computed from the year only)	A comparison or age calculation that is wrong on the birthday itself; the boundary value 60 was never tested	The patient who turns 60 on the day of the appointment does not get priority
I6	The developer assumed that only one doctor saves a prescription at a time	Prescription data is held in a place shared by simultaneous requests, so one request overwrites another	The PDF shows the details of the previous patient: a privacy breach and a risk of wrong medication

For I5 the case does not give the code, so the fault shown is the most probable one for the symptom described.

3. How the confusion in terminology made incident I8 possible

In I8 a refund test was marked "Pass" because the tester compared the output with a wrong expected value. Two things that the team's vocabulary did not separate are involved. First, the expected result (the oracle) and the actual output are different things, and the expected value itself can be wrong; the team had no concept of a defect in the test, only a "bug" in the software, so nobody treated the expected value as something to be reviewed. Second, a matching output was treated as proof of correctness, instead of as a result that is only as good as its oracle. Had the team had a defined test case (inputs, expected result and its source), the wrong value would have been traced to a rule document and the error would have been noticed before the verdict was recorded. The consequence is serious: a possible refund fault stayed hidden behind a false "Pass".

4. Does the team need a common glossary?

Yes, and the harm is concrete. If a patient report “the fee is wrong” is logged as a “bug”, the fix is made on the screen where it was noticed; the error (the misread GST rule) remains and the same fault can later appear in the invoice or the refund. Separating error, fault and failure lets the team fix the fault, find the error behind it and prevent the next one, and it makes the repository fields in Task 9 meaningful.

Table 13: Contents of a MediQueue glossary

Term	Why the team needs it	Entry must contain
Error, fault/defect, failure	Reports must say what went wrong and where	Definition and one MediQueue example each
Test case, oracle, test bed	Prevents I8: expected results need a source	Template for a test case with a “source of expected result” field
Verification and validation	Separates review work from running software	Which activity belongs to which phase
Severity and priority	Stops urgent and important being confused	Scales with examples (Task 8)
Defect origin and class	Allows the pattern analysis of Task 7	The list of origins and classes used in the repository
Release criteria	Replaces “the demo showed no errors”	Measurable exit conditions

TASK 5 – Evaluate the Application of Testing Principles 3 Marks
CO1 | Bloom's Level: K5 – Evaluate

The developers claim that “we tested everything and found no errors, so the software was correct”.

Which testing principles prove this claim wrong?

Discuss:

- **At least four software testing principles that apply to the MediQueue situation**
- **For each principle, the incident or decision in the case that violates or confirms it**
- **Which principle, if followed, would have had the largest effect on quality, and why**

Do not simply list principles. You are expected to relate each principle to evidence.

Evidence Expected
A justified evaluation of the developers’ claim using testing principles linked to evidence from the case.

Ans:

Which Testing Principles Prove the Developers’ Claim Wrong?

The claim “we tested everything and found no errors, so the software was correct” is wrong on several grounds. Seven principles apply to MediQueue, and for each I give the evidence from the case.

Table 14: Testing principles applied to MediQueue

Principle	Meaning in this case	Evidence in the case	Violated or confirmed
1. Testing shows the presence of defects, not their absence	A test with no failures does not prove correctness	The demo showed no errors, yet 60 defects were later reported by patients	Violated by the team
2. Exhaustive testing is impossible	Tests must be chosen by risk and technique	6,000 patients a day produce combinations (same-second bookings, ages, name lengths) that one demo cannot cover; no selection was made	Violated
3. Early testing	Defects in requirements and design should be sought early, where they are cheapest	52 of 100 defects arose in requirements and design; only 5 were found there. A production defect costs ₹40,000 against ₹500 at requirements	Violated; confirmed by the cost data
4. Defects cluster	A few areas hold most of the defects; look there first	I1 and I6 are both simultaneous-action defects; I2, I3 and I5 are all misread business rules	Confirmed; no one looked for clusters
5. Pesticide paradox	Running the same tests again stops finding new defects	The same demo scenario was the only "test"; it passed every time	Violated
6. Testing depends on context	A healthcare application with money and patient data needs more rigorous testing than a demonstration	I6 shows another patient's details; I3 mis-charges money. These were treated like screens in a demo	Violated
7. Absence-of-errors fallacy	A system with no failures can still fail the users' needs	Even if the demo had been free of errors, I2 (Sunday) and I10 (25-second screen) are requirement failures; the product did not meet the hospital's needs	Violated

The principle with the largest effect

Following **early testing** would have had the largest effect. The evidence is in the data: half the defects began in requirements and design, they were almost never found there (9.6 %), and each was 80 or 26.7 times more costly to fix in production than at its origin. In Task 3 I estimated that reviews alone would have saved about ₹8.5 lakh, which is roughly four times the whole cost of the defects that were found before release (₹2,10,500). It is also the principle whose violation affected the most incidents: I2, I3 and I10 originate in requirements and I1, I6 in design.

TASK 6 – Evaluate the Tester's Role 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

MediQueue had no independent tester.

Would an independent tester have changed the outcome, or would the same defects still have reached patients?

Discuss:

- **The role and responsibilities of a tester across the life cycle**
- **The advantages and limitations of developers testing their own code**
- **Which incidents a tester would most likely have caught, and at which stage**
- **The qualities a tester for a healthcare application must have**

Your answer must take a position and defend it.

Evidence Expected

A defended judgement on the value of an independent tester for MediQueue, linked to specific incidents.

Ans:

Would an Independent Tester Have Changed the Outcome?

My position is that an independent tester would have changed the outcome considerably, but would not have removed every defect. In my judgement the tester would probably have caught 5 of the 10 incidents, might have caught 4 more, and would not have caught 1. A tester is valuable because of what that person looks for and when, not because the person is a better programmer.

1. The role of a tester across the life cycle

Table 15: Responsibilities of a tester in each phase

Phase	Responsibility	MediQueue example
Requirements	Review for testability and missing rules; plan acceptance tests	Ask "which days does the outpatient department work?", "is the fee shown with GST?"
Design	Inspect for risks; design system tests; plan the test bed	Ask what happens when two requests arrive in the same second
Coding	Check unit tests and boundary values; review test cases	Ages 59/60/61; name lengths 40/41/42
Testing	Execute, record, report and analyse defects	Run the refund test with a verified expected value
Release and support	Decide with management whether exit criteria are met; analyse field defects	Do not release with an open critical defect such as I6

2. Advantages and limitations of developers testing their own code

Table 16: Developers testing their own code

Advantages	Limitations
Fast, with no hand-over and no extra staff	Blind to their own assumptions: the person who misread the GST rule will test with the same misreading
Knows the code and can debug at once	Tests to show it works, not to make it fail (testing as demonstration, TMM Level 1)
Good for unit tests of boundaries and logic	Rarely tests the specification, only the code; omissions (I2) are invisible
Cheapest place to fix coding defects (₹3,000)	Under release pressure, "testing" is shortened or skipped; no independent check on test cases (I8)

3. Which incidents a tester would have caught, and at which stage

Table 17: Likelihood that an independent tester would have caught each incident

Incident	Likely to be caught?	Stage and reason
I2	Yes	Requirements review: a testability question exposes the missing Sunday rule

Incident	Likely to be caught?	Stage and reason
I3	Yes	System test: compare the fee on the booking page with the amount charged
I4	Yes	Unit or system test: boundary value of 40 and 41 characters
I5	Yes	Unit test: ages 59, 60 and 61
I7	Yes	Documentation test: following the manual step by step
I1	Maybe	Only if a concurrency test was planned in design; needs a tool or script for same-second requests
I6	Maybe	As I1; requires a deliberate test of simultaneous saves
I8	Maybe	A tester can cause I8 as easily as a developer; the protection is a review of test cases and oracles
I10	Maybe	System test on a realistic network, but only if a response-time requirement exists
I9	No	The provider changed its key after release; only monitoring and configuration management can help

4. Qualities of a tester for a healthcare application

- A questioning attitude, and the willingness to prove the product wrong rather than right.
- Understanding of the hospital's rules (fees, GST, senior-citizen priority, working days) so the correct expected result is known.
- Attention to detail and to boundaries.
- Respect for patient privacy and confidentiality: test data must not contain real patient records.
- Communication skills, to report defects clearly to developers without blame and to management in terms of risk.
- Ability to judge risk, so that effort goes to money, privacy and safety first.

Conclusion: a tester would have found most of the defects that arise from missing rules, boundaries and documentation (I2, I3, I4, I5, I7), which is five of the ten incidents. Defects that need special tests (I1, I6, I10) and external changes (I9) require a plan, a test bed and monitoring as well, which is why the tester has to be part of a defined process (Task 2) and not a substitute for it.

TASK 7 – Trace the Origins of Defects 4 Marks
CO1 | Bloom's Level: K5 – Evaluate

Using the incidents reported after release, investigate where each defect originated.

Where in the life cycle are MediQueue's defects really coming from?

Discuss:

- **The origin of each incident: requirements, design, code, testing, documentation or environment**
- **The evidence in the case that supports each origin you assign**
- **Incidents that may have more than one origin, and how you decided**

- **What the overall pattern of origins tells the hospital about where to invest effort**

Where an incident could belong to more than one origin, argue for your choice.

Evidence Expected

A justified origin for each incident and an interpretation of the overall pattern of defect origins.

Ans:

Where Do MediQueue's Defects Come From?

I assign one origin to each incident, give the evidence from the case, and where a second origin is possible I argue for my choice. The pattern that emerges is that half the incidents arise before a line of code is written.

Table 18: Origin of each incident, with evidence and the argument for the choice

Incident	Origin	Evidence in the case	Possible second origin and why I chose this one
I1	Design	Two bookings in the same second get the same token. A token is by nature unique, so the need was obvious; what is missing is a design for safe allocation under simultaneous requests	Coding. I chose design because the failure follows from how token allocation was designed (read the last number, then add one), not from a slip in a statement
I2	Requirements	The case says the Sunday rule "was never written in the requirements"	None: the case itself names the origin
I3	Requirements	"The GST rule was misunderstood": the misunderstanding arose when the rule was captured	Design (two fee calculations). I chose requirements because even a perfect design would implement the wrongly understood rule
I4	Coding	The application crashes for names longer than 40 characters; the program does not validate input	Requirements (no maximum length stated). I chose coding because a program should reject long input rather than crash, whatever the specification says
I5	Coding	The rule fails exactly on the 60th birthday: a boundary comparison, typical of a coding slip	Requirements ("above 60" against "60 and above"). I chose coding on the assumption that the policy says 60 and over; if the written rule is ambiguous the origin moves to requirements
I6	Design	Prescription details of another patient appear when two doctors save together: data shared between simultaneous requests	Coding. Chosen as design because the concurrency behaviour should have been settled when the design said where the data lives
I7	Documentation	The manual describes a menu ("My Visits") that does not exist	None
I8	Testing	A test was passed against a wrong expected value: a defect in the test case, not in the product	None; the fault lies in the test
I9	Environment (external interface)	The failure is triggered by the SMS provider changing its key, an event outside the team's control	Design. I chose environment as the origin because the trigger is external, but the design lacks resilience and monitoring, which is where the fix lies
I10	Requirements	The case gives no response-time requirement; without a target the 25-second screen is never a failure on paper	Design or environment (Wi-Fi). Chosen as requirements because a missing non-functional requirement left nothing to test against

What the pattern of origins tells the hospital

Table 19: Pattern of origins compared with the defect data

Origin	Incidents	Share of the ten incidents	Defects introduced in the data (of 100)	Found in own phase
Requirements	I2, I3, I10	30 %	30	2 (6.7 %)
Design	I1, I6	20 %	22	3 (13.6 %)
Coding	I4, I5	20 %	38	15 (39.5 %)
Testing	I8	10 %	5	—
Documentation	I7	10 %	5	0
Environment	I9	10 %	not in the data table	—

- **Half the incidents (I1, I2, I3, I6, I10) and 52 % of the data originate in requirements and design.** The incident sample and the defect data agree, which strengthens the conclusion.
- **Detection is worst where origin is greatest.** The team found 39.5 % of coding defects in their own phase, but only 13.6 % of design and 6.7 % of requirements defects. Developers can see code, but nobody was looking at the requirements and the design.
- **Investment advice:** put the first effort into requirement and design verification (reviews with hospital staff, a design inspection), then into testing practices for the coding phase. Hiring more people to "test at the end" would pay back less, because by then most of the defects have already been built into the product.

TASK 8 – Evaluate Defect Classes and Severity 4 Marks
CO1 | Bloom's Level: K5 – Evaluate

A defect classification scheme helps the team understand the nature of its defects.

Which classes of defects are most harmful to patients, and do the team's priorities reflect that?

Discuss:

- **A suitable defect classification (requirements/specification, design, coding, testing defects and their sub-classes)**
- **The class, severity and priority of each incident, with reasons**
- **Why severity and priority may differ for the same defect (for example, I7 and I1)**
- **Which defect class the team should target first, and why**

Simply placing defects in a table is not sufficient. You are expected to justify each classification.

Evidence Expected
A justified classification of all ten incidents by class, severity and priority, with a recommendation on which class to target first.

Ans:

Which Defect Classes Are Most Harmful to Patients?

I use a classification with four classes and their sub-classes. I then classify each incident by class, severity and priority, explain why severity and priority can differ, and recommend which class to target first.

Table 20: Defect classification scheme used for MediQueue

Class	Sub-classes	MediQueue examples
Requirements / specification defects	Functional description (missing, wrong, ambiguous); feature and feature interaction; non-functional requirements; interface description	I2, I3, I10
Design defects	Algorithm and processing; control, logic and sequence (including concurrency); data and shared resources; module and external interface	I1, I6, I9
Coding defects	Control flow and comparison; data and input validation; initialisation; interface use; code and user documentation	I4, I5, I7
Testing defects	Test case design and expected results (oracle); test procedure; test environment and test harness	I8

I7 is a defect in the user manual; because no separate documentation class exists in this scheme, I record it under the documentation sub-class of coding-phase work products.

Table 21: Severity and priority scales used

Level	Severity (effect on the patient and the hospital)	Priority (order of repair)
Critical / P1	Risk to patient safety or privacy, or wrong money for many patients	Fix immediately; hot-fix outside the normal release
High / P2	Core function wrong or unusable for some patients; money or trust affected	Fix in the next release
Medium / P3	Function degraded; workaround exists	Plan within a few releases
Low / P4	Cosmetic or information only	Fix when convenient

Table 22: Class, severity and priority of each incident, with reasons

Incident	Class and sub-class	Severity	Priority	Reasons
I1	Design: control, logic and sequence (concurrency)	High	P2	Two patients with one token cause disputes and a wrong queue order. Serious, but it needs two bookings in the same second for the same doctor, and the front desk can re-issue a token
I2	Requirements: functional description (missing rule)	Medium	P2	Patients travel to a closed department; easily corrected once the rule is written
I3	Requirements: functional description (misread rule)	High	P1	Wrong amount charged to every patient affected: direct money loss, complaints and legal risk
I4	Coding: data and input validation	High	P3	A crash stops the booking, but only for names over 40 characters and the patient can shorten the name
I5	Coding: control flow / boundary comparison	Medium	P3	The priority is lost only on the birthday; the patient is not harmed medically

Incident	Class and sub-class	Severity	Priority	Reasons
I6	Design: data and shared resources (concurrency)	Critical	P1	Another patient's details on a prescription: privacy breach and risk of wrong medication
I7	Documentation	Low	P1	No loss of function, but every patient who tries to cancel is misled; the fix is a text change and takes minutes
I8	Testing: test case and oracle	Medium	P2	Does no harm by itself but hides a possible refund fault and gives false confidence
I9	Design: external interface (trigger from the environment)	Medium	P1	Reminders stop for all patients until the key is replaced; the fix is a configuration change
I10	Requirements: non-functional (performance)	Medium	P3	Doctors wait but can work; needs analysis and a target before it can be fixed

Why severity and priority differ: I7 and I1

- **Severity measures the effect of the defect; priority measures the order of repair.** Priority combines severity with how often the defect occurs, whether a workaround exists and what the fix costs.
- **I7 has low severity but high priority:** nobody is harmed and no function is lost, but all patients who try to cancel are misled and call the helpdesk. Correcting a line of text costs almost nothing, so it should be done at once.
- **I1 has high severity but medium priority:** the effect is serious (the queue is wrong) but it needs two bookings in the same second for the same doctor, the front desk can re-issue a token, and the proper fix is a design change that needs time to do properly. It still belongs in the next release.

Which classes are most harmful to patients, and do the team's priorities reflect this?

The most harmful incident, I6, is a design defect, and the next most serious (I3, I1) come from requirements and design. The coding defects (I4, I5) are visible but cause less harm. The team's priorities do not reflect this: its effort, such as it was, went to coding (the only phase with a developer self-check), while requirements and design received no review at all. The team had no severity or priority scheme; defects were handled as complaints arrived.

Class to target first

I recommend that the team targets **requirements and specification defects first**. They are the largest class in both the incidents (3 of 10) and the data (30 of 100), 28 of the 30 escaped their own phase, and they feed the later classes, because a design or code that faithfully follows a wrong requirement is wrong however well it is built. I would repair I6 at once as a separate hot-fix, because it is a privacy breach, and follow it with design inspection (Task 10).

TASK 9 – Design and Appraise a Defect Repository 3 Marks
CO1 | Bloom's Level: K5 – Evaluate

The hospital currently records complaints in e-mails and a spreadsheet.

What information must a defect repository hold so that the team can learn from its defects and stop repeating them?

Discuss:

- **The fields a defect repository for MediQueue must contain, and why each is needed**
- **How the repository supports defect analysis, process improvement and prevention**
- **The weaknesses of the current e-mail and spreadsheet approach**

You are expected to justify your repository design, not just list fields.

Evidence Expected

A defect repository design for MediQueue with a justification for each field and an evaluation of its benefits.

Ans:

What Must a Defect Repository Hold?

The hospital currently logs complaints in e-mails and a spreadsheet. That is enough to answer "what was reported?" but not "where do our defects come from and what should we change?". A repository must therefore hold not only what the patient saw, but also where the defect came from, what it cost and how it was prevented from recurring.

1. Repository fields and why each is needed

Table 23: Defect repository design for MediQueue

Field group	Fields	Why it is needed
Identification	Defect ID, title, date and time, reporter and source (patient, staff, tester, review, monitoring)	A unique ID prevents duplicates and allows links; the source shows which activity finds defects
Description	Steps to reproduce, expected result, actual result, screenshots, app version, device, network	Lets a developer reproduce the failure; separates failure from fault (Task 4)
Classification	Origin phase, defect class and sub-class, module (for example booking, payment, prescription)	Enables the pattern analysis of Tasks 7 and 8 and shows which module clusters defects
Impact	Severity, priority, number of patients affected, safety / privacy / money flag	Decides repair order; the privacy flag identifies the defects that matter most
Detection	Phase in which it was detected, technique or activity that found it, who found it	Gives phase containment (Table 8) and shows which reviews and tests earn their cost
Resolution	Owner, status, fix description, release in which fixed, who verified, regression test added	Tracks every defect to closure and prevents a fix being lost
Cost	Hours to find and fix, cost, patients affected	Allows the cost comparison of Task 3 to be made from real data
Root cause and prevention	Error behind the fault (misread rule, missing check), prevention action, related process change	Turns individual defects into lessons and checklist items
Links	Requirement ID, test case ID, related defects, external ticket number	Traceability: shows which requirement or test was involved

2. How the repository supports analysis, improvement and prevention

Table 24: Benefits of the repository

Purpose	What the repository allows	Example for MediQueue
Defect analysis	Counting and grouping by origin, class, module and severity; Pareto charts	A query by origin gives the pattern of Table 19 automatically and shows that half the defects come from requirements and design
Process improvement	Measuring the effect of a change from release to release	After introducing requirement reviews, the share of defects found at requirements should rise from 6.7 %; if it does not, the reviews need change
Prevention	Using the root-cause field to build checklists and training	Two concurrency defects (I1, I6) justify a concurrency item in the design checklist

3. Weaknesses of the e-mail and spreadsheet approach

Table 25: E-mails and spreadsheet compared with a repository

Aspect	E-mails and spreadsheet	Repository
Uniqueness	The same complaint is reported several times; no ID	One record per defect with a unique ID
Standard fields	Free text; origin, class and cost are not recorded	Defined fields that can be counted
Workflow	No status or owner; closed defects are not tracked	Status, owner and verification are mandatory
Analysis	Manual counting, error-prone and slow	Filters and reports
Links	No link to requirements, tests or releases	Traceability
History	Versions overwritten or lost	Audit trail of every change
Privacy	Patient details travel in e-mails and copied sheets	Access controlled; patient data masked

TASK 10 – Evaluate Defect Examples and Prevention Strategies 4 Marks
CO1 | Bloom's Level: K5 – Evaluate

Select three incidents that represent different defect classes.

For each selected defect, what single change to the process would have prevented it?

Discuss:

- **The root cause of each selected defect**
- **A prevention strategy for each (for example, requirement reviews, design inspection, coding standards, test oracles)**
- **The cost and effort of each strategy compared with its benefit**
- **Which strategy gives the best value for the hospital**

Your recommendation must be supported by evidence from the case and the defect data.

Evidence Expected
A root-cause analysis of three defects with justified, cost-aware prevention strategies.

Ans:

Prevention Strategies for Three Defects from Different Classes

I choose three incidents from three different classes: I3 (requirements), I6 (design) and I5 (coding). For each I give the root cause and the single process change that would have prevented it, then compare the costs and benefits of the three changes.

Table 26: Root cause and prevention of the three chosen defects

	I3: fee ₹500 shown, ₹550 charged	I6: previous patient's prescription	I5: priority missing at age 60
Class	Requirements	Design	Coding
Root cause	The GST rule was understood differently by different people and never written with a worked example	Concurrency was not considered: data for the PDF is shared between simultaneous requests	No test or review of the boundary value 60; comparison written without checking the policy wording
Prevention strategy (single change)	Requirement review with the accounts staff, with a worked fee example (₹500 → ₹550) in every fee requirement	Design inspection with a concurrency and shared-data checklist; one prescription object per request	Unit tests with boundary values 59, 60 and 61 and peer code review against a coding standard
Cost if the defect is found there (₹)	500 (requirements)	1,500 (design)	3,000 (coding)
Cost as found (₹)	40,000	40,000	40,000
Saving for this defect (₹)	39,500 (98.8 %)	38,500 (96.3 %)	37,000 (92.5 %)

Cost and effort of each strategy compared with its benefit

A strategy covers every defect of its kind, not only the one chosen, so I compare the cost of each activity (hours at an assumed ₹500 per hour) with the saving it would bring across the defect data. Each saving is calculated with the same flow model as Table 10, changing one phase at a time: requirement review at 60 % efficiency, design inspection at 60 %, unit tests with peer review at 55 %.

Table 27: Cost and benefit of each strategy applied alone

Strategy	Effort per release	Cost (₹)	Saving in the 100-defect release (₹)	Benefit : cost	Defects it addresses
Requirement review with worked examples	24 hours	12,000	3,91,145	32.6 : 1	I2, I3, I10
Design inspection with concurrency checklist	32 hours	16,000	6,73,465	42.1 : 1	I1, I6, I9
Boundary unit tests and peer code review	60 hours	30,000	9,03,817	30.1 : 1	I4, I5

- **Best value: design inspection (42 : 1).** It is cheap, addresses the only critical incident (I6) and also removes requirement defects that escaped the first review, because the design is checked against the requirements.
- **Requirement review is next (33 : 1)** and the cheapest in absolute terms (₹12,000). Because it comes first in the life cycle, it makes the design inspection more effective.
- **Unit tests and code review give the largest absolute saving (₹9 lakh)** because 38 defects arise in coding, but the ratio is the lowest because they need the most effort.

- The savings overlap and cannot be added: the same defect cannot be saved twice. The combined effect is calculated in Table 29.
- The results depend on the assumed efficiencies and the rate of ₹500 an hour; they should be replaced by the hospital's own figures when the repository has collected them.

TASK 11 – Recommend and Defend a Quality Improvement Plan 4 Marks
CO1 | Bloom's Level: K5 – Evaluate

Based on your analysis in Tasks 1–10, recommend a quality improvement plan for MediQueue.

Discuss:

- **The process changes you recommend and their order of priority**
- **The role of testers, reviews and the defect repository in your plan**
- **The expected effect on the number and cost of defects**
- **Trade-offs, risks and limitations of your plan**
- **Whether the plan remains practical for a small team of five developers**

Your recommendation must follow from your own analysis. A generic list of best practices is not sufficient.

Evidence Expected
A technically justified quality improvement plan supported by the evidence developed in earlier tasks.

Ans:

A Quality Improvement Plan for MediQueue

The plan follows from the evidence of Tasks 1 to 10: half the defects begin in requirements and design (Task 7), they are found too late at 80 times the cost (Task 3), and the team has no tester, no review and no repository. I list the changes in order of priority, say who does what, estimate the effect, and test whether a team of five developers can carry it.

1. Process changes in order of priority

Table 28: The improvement plan in order of priority

Priority	Change	Owner	Effort per release	Incidents and defects targeted
0 (now)	Hot-fix I6 and I3; add an alert when the SMS key fails (I9)	Developers	Within two weeks	I6, I3, I9
1	Requirement review with a checklist, worked examples and a list of non-functional requirements (working days, field limits, response time)	Tester leads; hospital staff and two developers review	24 h	I2, I3, I10; 30 requirement defects
2	Design inspection with concurrency, shared-data and external-interface checklists	Tester leads; two developers	32 h	I1, I6, I9; 22 design defects
3	Unit tests with boundary values, peer code review and a coding standard	Developers	60 h	I4, I5; 38 coding defects

Priority	Change	Owner	Effort per release	Incidents and defects targeted
4	Written test plan; test cases with expected results from a named source, reviewed by a second person; system tests on a test bed with concurrency and Wi-Fi tests; check of the manual	Tester	60 h design, 80 h execution	I1, I6, I7, I8, I10
5	Defect repository (Table 23) and a monthly analysis of origins, classes and cost	Tester	12 h	All; basis for improvement

2. The role of testers, reviews and the repository

- **The tester** (a dedicated person, which is my role as Quality Engineer) owns the test plan, leads the reviews and keeps the repository. The tester takes part from the requirements phase, not only at the end.
- **Reviews** are held for requirements and design with the people who know the rules (accounts, front desk) and the people who build. They are cheap (₹500 and ₹1,500 per defect found).
- **The repository** records every defect with origin, class and cost, so that each month the team can see whether the changes work and where to improve next.

3. Expected effect on the number and cost of defects

I assume review efficiencies of 60 % for requirements and design, 55 % in coding (unit tests plus peer review), 60 % for independent system testing and 80 % for the documentation check, applied to defects present in each phase. For comparison, the present figures were 6.7 %, 6.0 %, 17.6 % and 26.7 %.

Table 29: Defect flow and cost with the full plan

Phase	Present	Detected	Cost per defect (₹)	Cost (₹)	Escaping
Requirements review (60 %)	30	18.0	500	9,000	12.0
Design inspection (60 %)	34.0	20.4	1,500	30,600	13.6
Coding: unit tests and peer review (55 %)	51.6	28.4	3,000	85,140	23.2
System testing (60 %)	28.2	16.9	8,000	1,35,456	11.3
Documentation check (80 %)	5	4.0	500 (assumed)	2,000	1.0
Production	12.3	12.3	40,000	4,91,520	—
Total		**100**		**7,53,716**	

- **Defects reaching patients fall from 60 to about 12** (an 80 % reduction).
- **The cost of defects falls from ₹26,10,500 to ₹7,53,716**, a gross saving of ₹18,56,784 (71 %).
- **The plan costs about 268 hours** (24 + 32 + 60 + 60 + 80 + 12), which at ₹500 an hour is ₹1,34,000. The net saving is therefore about **₹17,22,800** per release.

4. Trade-offs, risks and limitations

- The efficiencies are assumptions; the real figures may be lower at first. If every efficiency were only half as high, the saving would still be large, but the target should be reviewed after three releases using the repository.
- Reviews and a tester add about 5.6 % to the effort of the eight-month build (268 of 4,800 hours) and add some days before release. The hospital must accept a later release in return for fewer failures.
- One tester is a single point of failure; a developer must be trained to cover.
- There is a risk that reviews turn into a formality; the repository shows whether they find defects.
- The plan does not address production monitoring beyond an SMS alert, nor load testing at the full 6,000 patients a day.

5. Is the plan practical for five developers?

Yes. The 268 hours are shared: the tester carries 152 hours (planning, test design, execution and repository) and the five developers share the 116 hours of reviews and unit testing, about 23 hours each over eight months. This is under three working days per developer. The plan buys 80 % fewer production defects for about 5.6 % extra effort.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Quality Process 8 Marks
CO1 | Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital now wants to release a new version of MediQueue every two weeks instead of every six months.

At the same time, MediQueue must connect to the national digital health ID system, which is maintained by an external agency.

Patient health records will now be shared between hospitals.

Re-evaluate your original solution in response to these changed conditions.

You may: RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis – 2 Marks
CO1 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed from the original situation**
- **Which assumptions in your quality improvement plan are affected**
- **Why frequent releases change the role of testing and the tester**
- **Why sharing health records raises the severity of some defect classes**

Evidence Expected
A clear analysis of how the changed conditions affect the original quality improvement plan.

Ans:

Impact of the Changed Conditions

Three things change at the same time: the release rhythm, the dependence on a system the hospital does not control, and the sensitivity of the data. My plan in Task 11 was designed for a six-month release by a closed team handling data of one hospital, and several of its assumptions no longer hold.

Table 30: What has changed and its effect on the plan

Aspect	Original situation	Changed situation	Effect on the quality plan
Release frequency	One release in six months; time to test and stabilise	A release every two weeks: 13 times as many releases	Every review, test and decision must fit in ten working days; manual regression cannot be repeated each time
Ownership of code	Everything built and controlled by the team	The national digital health ID system is maintained by an external agency	New defect origins that the team cannot fix; need for contract tests and for tracking defects with an outside party
Data	Appointments and prescriptions inside one hospital	Patient health records shared between hospitals	A defect can expose or corrupt the records of patients at other hospitals; legal and privacy duties increase

Assumptions in my plan that are affected

- **Time:** the plan assumed a stable period for system testing (80 hours of manual execution per release). With ten working days this is impossible.
- **Control:** the plan assumed all defects are in code the team can change; external defects need a different route.
- **Impact:** the severity scale assumed that a defect harms the patients of one hospital.
- **Capacity:** the effort of 268 hours was set for a release every six months; I test it against a two-week capacity in Part B.
- **Repository:** it had no field for the owner of a defect, the external system or the version of an interface.

Why frequent releases change the role of testing and of the tester

With a release every two weeks, there is no longer a testing phase at the end; testing becomes a continuous part of each cycle. The tester can no longer execute test cases by hand after development: the tester must plan risk-based tests for each change, maintain automated regression and contract tests, take part in reviews on the first day, and give the release decision against defined exit criteria. Testing's purpose changes from finding all defects before a large release to keeping each small release safe and learning quickly from each one.

Why sharing health records raises the severity of some defect classes

Table 31: Severity of defect classes before and after record sharing

Defect type (example)	Severity before	Severity now	Reason
Concurrency or shared-data defect (I6)	Critical (one stranger's PDF)	Critical, with legal consequences	The same defect could now show another hospital's patient data and wrong history, and a privacy breach would be reportable
Token or identity duplication (I1)	High	Critical	If identifiers are duplicated, records may be attached to the wrong person, so treatment decisions rest on the wrong history
External interface failure (I9)	Medium	High	A failed or changed interface can block patient look-up or send data to the wrong place
Rule or calculation defect (I3)	High	High	Unchanged: money, but not a risk to the shared data
Input and boundary defects (I4, I5)	High / Medium	High / Medium	Unchanged unless the invalid data is passed on to other hospitals

B. Evaluation of the Original Plan – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Evaluate whether your recommended plan remains appropriate.

Discuss:

- **Whether the planned reviews and test activities fit into a two-week cycle**
- **Which new defect origins are introduced by the external health ID system**
- **Whether your defect repository is still sufficient**

Evidence Expected

A technically supported evaluation of the suitability of the original plan under the new conditions.

Ans:

Is the Original Plan Still Appropriate?

Most of the activities remain valid, but the plan does not fit the two-week cycle as written and it does not cover the external system. I test it against capacity.

1. Do the reviews and tests fit into a two-week cycle?

A two-week cycle has 10 working days. With five developers at 6 productive hours a day the team has 5 × 10 × 6 = 300 hours, and 360 hours together with the tester (60 hours). I assume that the quality work must not exceed 20 % of capacity, which is 72 hours. I assume that review, design and test-design effort grows with the size of a change (one thirteenth of a six-month release), while the 80 hours of manual regression must be repeated in full each time, and that testing the external interface and privacy by hand takes about 30 hours.

Table 32: Quality effort per two-week release: original plan against redesigned plan

Activity	Original plan, per six-month release (h)	Original plan repeated every two weeks (h)	Redesigned plan, per two-week release (h)
Requirement and change review	24	1.8	8

Activity	Original plan, per six-month release (h)	Original plan repeated every two weeks (h)	Redesigned plan, per two-week release (h)
Design and contract check	32	2.5	6
Unit tests and peer code review	60	4.6	10
Test design for the change	60	4.6	4
Full regression (manual / automated run and analysis)	80	80.0	4
Contract tests with the health ID system, and privacy and security checks	—	30.0 (manual)	14
Exploratory and risk-based testing	—	—	8
Repository, retrospective and go/no-go	12	0.9	6
Total	**268**	**124.4**	**60**
Share of the 360 h available	—	34.6 %	16.7 %
Against the 72 h budget	—	1.7 times the budget	Within the budget

- The original plan, repeated every two weeks, takes 124 hours, 34.6 % of the team's time and 1.7 times the budget. The cause is the manual regression run (80 hours), which does not shrink when the change is small. The plan therefore does not fit.
- The redesigned plan takes 60 hours (16.7 %). It needs an investment of about 150 hours to automate the regression suite and the contract tests (100 and 50 hours). If this is spread over the first three cycles, each of them carries 110 hours (30.6 %), which is about 38 hours above the budget; the feature scope of those three releases should therefore be cut by about one tenth.

2. New defect origins introduced by the external health ID system

- **Contract mismatch:** the team's understanding of the interface differs from that of the agency (a requirement defect shared by both sides).
- **Undocumented change:** the agency changes a format or key without notice, which is the pattern of incident I9.
- **Availability and performance:** the external service is slow or unavailable, and this affects booking.
- **Environment difference:** the test (sandbox) version of the service behaves differently from the live version.
- **Data quality:** wrong, duplicate or incomplete records arrive from outside and are shown to doctors.
- **Privacy and consent:** shared records are shown to a person who should not see them.

3. Is the repository still sufficient?

No. The repository of Task 9 would need these fields in addition: owner of the defect (our team, external agency, joint), external ticket number, interface or API version, a data-exposure flag (was patient data revealed or altered?), release number, how the defect was detected (review, test, monitoring or complaint), a workaround, and the time taken to resolve it. Without them the team

cannot tell which defects it can fix and which it must escalate, and it cannot show that the shared-data defects were handled in time.

C. Adaptation or Redesign – 2 Marks
CO1 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the quality process.

If you modify or redesign it:

- **Describe the revised process and the tester's revised role**
- **Explain which testing principles now become more important**
- **Explain how defects from the external system will be classified and tracked**

If you retain the original plan, provide sufficient justification for why no change is necessary.

Evidence Expected
An appropriate response to the changed requirement with clear quality-engineering reasoning.

Ans:

Decision: Modify the Quality Process

I decide to **modify** the plan. The principles that the evidence supports (verify early, a tester involved from the start, a defect repository, boundary and concurrency tests) remain, but how they are carried out must change. I do not retain the plan, because it needs 1.7 times the budget and does not address the external system. I do not redesign it completely, because all six activities of the original plan carry over in lighter form (Table 32).

1. The revised process

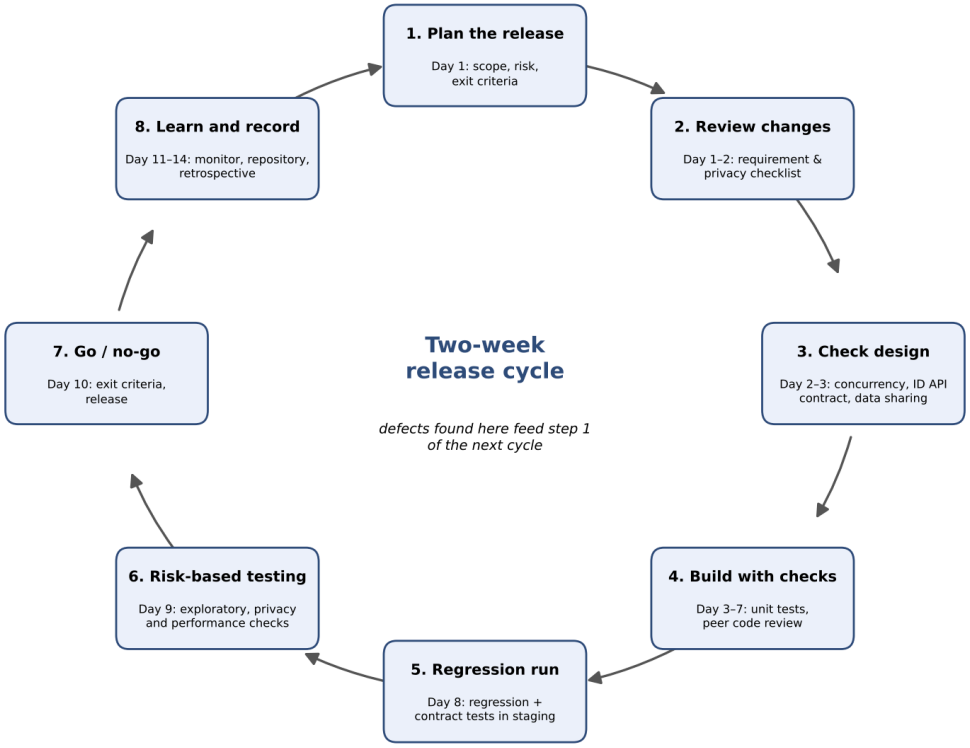


Figure 2: Revised eight-step two-week release cycle

- **Reviews become change-based:** only the requirements and design that change in the release are reviewed, using the checklists of Task 11 plus a privacy and interface item.
- **Regression is automated:** the stored tests run in the pipeline on every build; a human reads only the failures.
- **Contract tests:** a stub of the health ID service is used for repeatable tests of the interface, and a short test against the agency's sandbox is run in each cycle.
- **Exit criteria:** no open critical defect; no unreviewed change that touches patient data; regression passing; contract tests passing.

2. The tester's revised role

- Owns the regression and contract tests and keeps them up to date each cycle.
- Chooses tests by risk for each release, giving first place to patient data and identity.
- Takes part in planning on day 1 and gives the go/no-go advice on day 10.
- Is the contact for the external agency on defects and interface changes, with the repository as the record.

3. Testing principles that now become more important

- **Early testing:** a review on day 1 of the cycle is the only time that fits into ten days.
- **Exhaustive testing is impossible:** testing every combination in each cycle is not possible; tests must be chosen by risk.

- **Defects cluster:** the external interface and the modules that change most in a cycle are where to look.
- **Pesticide paradox:** automated tests that stay the same stop finding new defects, so the suite is updated in every cycle from the new defects in the repository.
- **Testing depends on context:** shared health data needs stricter privacy testing than an appointment screen.
- **Absence-of-errors fallacy:** passing contract tests do not show that the whole record exchange is correct for the hospitals.

4. Classification and tracking of defects from the external system

Table 33: Classification and tracking of defects involving the external health ID system

Category	Example	Who fixes it	Handling and tracking
E1: Our misuse of the interface	We send the key in the wrong format	Our team	Normal defect (design or coding, external interface); fix, add contract test
E2: External service defect or outage	The service times out or returns wrong data	External agency	Defect logged with the agency's ticket number and a time clock; a fallback (for example booking without the ID, flagged for later matching) keeps the service running
E3: Contract ambiguity or undocumented change	The agency changes a response format without notice	Both	Treated as a joint requirement defect; agree the contract in writing; record the interface version
E4: Data-quality defect in shared records	Duplicate or wrong patient record from outside	External agency and hospital	Data-exposure flag set; patient-safety review; severity at least High
E5: Environment difference	The sandbox accepts what the live service rejects	Both	Add a live-like check to the release; record which environment showed the failure

All defects, whoever owns them, are given a severity by their effect on patients and a priority by the effect and by the time the owner needs to repair them. They stay open in the repository until the effect has been removed (by a fix or a workaround), and they are reviewed every month with the agency.

D. Evidence-Based Defence – 2 Marks
CO1 | Bloom's Level: K5 – Evaluate

Defend your final decision using evidence.

Use:

- **The defect data and cost figures**
- **Defect origins and classes**
- **Release frequency**
- **Patient-safety and data-privacy risk**
- **Team size and capacity**

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final decision integrating defect evidence, process reasoning and the changed constraints.

Ans:

Defending the Decision to Modify the Plan

Table 34: Evidence supporting the decision

Evidence	What it shows	Consequence for the decision
Defect data and cost	60 % of the defects reached patients and caused 91.9 % of the cost (₹24,00,000 of ₹26,10,500). Each production defect costs ₹40,000, which is 80 times a requirement defect	The value of finding defects early is unchanged, so the review and test activities stay
Defect origins and classes	Half of the defects come from requirements and design (52 of 100); concurrency (I1, I6) and rules (I2, I3, I5) are clusters	Keep change-based reviews and the concurrency checklist; extend them to the external interface
Release frequency	13 releases for each of the old ones; the original plan would need 124 hours per cycle (1.7 times the budget of 72 hours)	The way the plan is carried out must change: automation and change-based reviews bring it to 60 hours
Patient safety and privacy	Shared records raise the severity of I6 and I1 to the level of legal and safety risk; the interface adds origins E1 to E5	Add privacy, contract and exit-criteria checks and a data-exposure field
Team size and capacity	Five developers and a tester give 360 hours per cycle; the redesigned plan needs 60 hours (16.7 %), plus 150 hours of one-off automation	Practical, if the feature scope of the first three cycles is reduced by about a tenth

Why not retain, and why not redesign?

- **Not retain:** the plan as it stands needs 124 hours per cycle against a budget of 72 hours and has no means of dealing with the external system or with the shared data.
- **Not redesign from the beginning:** the evidence for early verification, an independent tester and a repository is stronger, not weaker, after the change, and all six activities of the original plan carry over.

The effect on the cost of defects

If the same defect rate per change is assumed, the old way of working would let about 4.6 defects reach patients in each two-week release (100 defects ÷ 13 releases × 60 % escape), at a cost of about ₹1,84,600 in fixes per release. With the plan the figure is about 0.95 defects, or ₹37,800 per release. These figures leave out the harm that the shared data makes possible, which is the main reason for the extra privacy and contract checks.

Risks accepted and review trigger

- The efficiencies (60 %, 55 %) and the capacity figures are assumptions; I will replace them with the hospital's own figures from the repository after the first three releases.
- The one-off automation work lowers the feature output of the first three releases by about a tenth.
- The external agency is outside the hospital's control; the fallback in E2 reduces but does not remove the risk.

- I would reopen the decision, and consider a redesign, if more than one defect per release reaches patients, if quality work takes more than 20 % of the capacity, if any privacy incident occurs, or if the agency changes the interface with less than two weeks' notice.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Evaluate Testing as an Engineering Activity	5
Task 2 – Evaluate the Role of Process in Software Quality	2
Task 3 – Evaluate Testing as a Process	6
Task 4 – Apply and Critique Basic Definitions	4
Task 5 – Evaluate the Application of Testing Principles	3
Task 6 – Evaluate the Tester's Role	3
Task 7 – Trace the Origins of Defects	4
Task 8 – Evaluate Defect Classes and Severity	4
Task 9 – Design and Appraise a Defect Repository	3
Task 10 – Evaluate Defect Examples and Prevention Strategies	4
Task 11 – Recommend and Defend a Quality Improvement Plan	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	50